305432 เครือข่ายเซนเซอร์ (Sensor Networks)

# Publish-Time Optimizations for Web-Based Visualizations

การเพิ่มประสิทธิภาพเว็บแสดงภาพข้อมูลเชิงโต้ตอบด้วยการประมวลผลล่วงหน้าช่วงเผยแพร่

Ron Pechuk, Jeffrey Heer
University of Washington

2025 IEEE Visualization and Visual Analytics (VIS), pp. 226–230

IEEE Doc ID: 11298742 DOI: 10.1109/VIS60296.2025.00051

**นำเสนอโดย นายธีรภัทร ภู่ระย้า รหัสนิสิต 66362416**

# ที่มา ปัญหาความหน่วง และตัวชี้วัด

## ข้อจำกัดของการประมวลผลตอนรันไทม์

### 1 การคำนวณซ้ำซ้อนบนเบราว์เซอร์

เบราว์เซอร์ต้องดาวน์โหลดข้อมูลดิบ รันคิวรีรวมกลุ่ม และวาดชาร์ตใหม่ทุกครั้ง
ส่งผลให้เกิดความหน่วงสูง โดยเฉพาะกับชุดข้อมูลขนาดใหญ่

### 2 ปัญหาความหน่วงตอนเปิดหน้าครั้งแรก

ระบบแคชแบบเดิมทำงานตอนรันไทม์ ทำให้ผู้เข้าชมคนแรก (Cold-Start)
ยังคงต้องรอโหลดข้อมูลและคำนวณใหม่ตั้งแต่ต้นโดยไม่ได้รับประโยชน์จากแคช

### 3 ภาพปรากฏแต่ยังไม่พร้อมโต้ตอบ

ภาพปรากฏแล้ว แต่ยังต้องเริ่มคอมโพเนนต์และตัวกลางคิวรี
ระบบจึงยังรับการคลิก ซูม หรือการเลือกช่วงข้อมูลไม่ได้

## ตัวชี้วัดประสิทธิภาพและต้นทุนระบบ

### 1 TTR (Time-to-Render) · เวลาจนภาพปรากฏ

เวลาตั้งแต่เริ่มส่งคำขอ จนกระทั่งชาร์ตแสดงผลครบถ้วนบนหน้าจอ
สะท้อนความเร็วที่ผู้ใช้รับรู้ได้จริง (Perceived Latency) เทียบเคียงกับ FCP

### 2 TTA (Time-to-Activation) · เวลาจนพร้อมโต้ตอบ

เวลาจนกระทั่งระบบพร้อมรับการคลิก ซูม หรือเลือกฟิลเตอร์ข้อมูล
ชี้วัดความพร้อมใช้งานจริงของอินเทอร์เฟซ เทียบเคียงกับ TTI บนเว็บ

### 3 SC (Storage Cost) · ขนาดทรัพยากรที่เผยแพร่

ขนาดทรัพยากรทั้งหมดที่เผยแพร่ รวมข้อมูลและผลคำนวณล่วงหน้า
เปรียบเทียบพื้นที่กับ Baseline เพื่อประเมินต้นทุนของการลดเวลา

ที่มา: Sections 1 และ 3.1 (หน้า PDF 1–2)

# สถาปัตยกรรม Mosaic Publisher

## สถาปัตยกรรมไปป์ไลน์ช่วงเผยแพร่

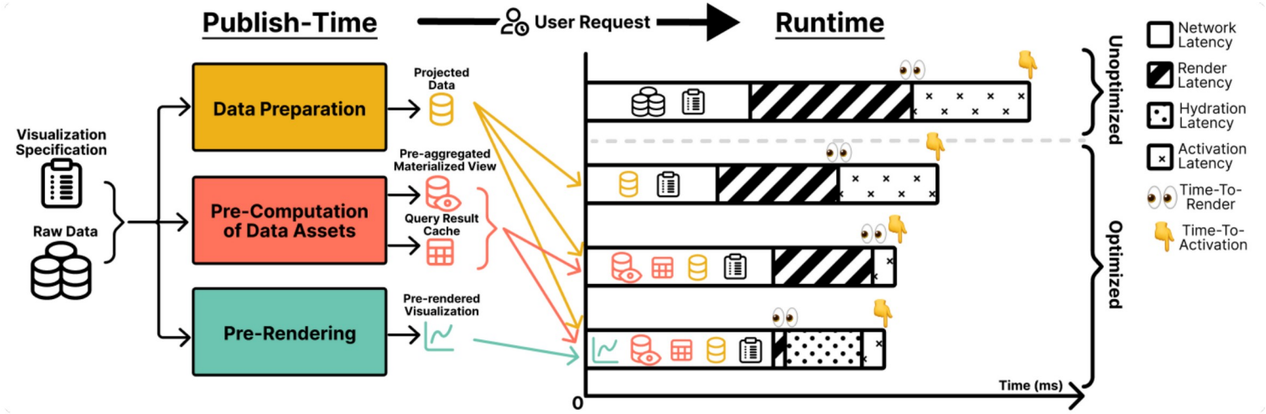



Figure 1: การแบ่งกลุ่มงาน 3 ส่วนที่ย้ายไปทำล่วงหน้าก่อนผู้ใช้ส่งคำขอ (หน้า PDF 1)

## ขั้นตอนการประมวลผลของ Publisher

**1. วิเคราะห์ AST & จำลอง Virtual DOM** → **2. คำนวณคิวรีล่วงหน้าผ่าน DuckDB** → **3. Rewrite AST & สร้าง JS Bundle**

สรุปกลไกการทำงานจาก Sections 4.1–4.3 (หน้า PDF 3)

## ชิ้นส่วนผลลัพธ์ส่งมอบให้ไคลเอนต์

### 1 ข้อมูลจัดระเบียบพร้อมใช้

กรองเฉพาะคอลัมน์ที่จำเป็นและแปลงข้อมูล
ลดภาระการส่งข้อมูลข้ามเครือข่าย

### 2 ผลคิวรีสำเร็จรูป

เติมแคชและฐานข้อมูล DuckDB-WASM
ล่วงหน้าผ่าน Mosaic Coordinator

### 3 ภาพเริ่มต้นและชุดสคริปต์

ส่งภาพเรนเดอร์สำเร็จให้แสดงผลทันที
พร้อมสคริปต์เปิดการโต้ตอบตามหลัง

ที่มา: Sections 3.2–3.4 และ 4.1–4.3 (หน้า PDF 2–3)

# ภาพล่วงหน้าและต้นทุน Client Hydration

## การแยกสัดส่วนความหน่วงและคอขวด Hydration

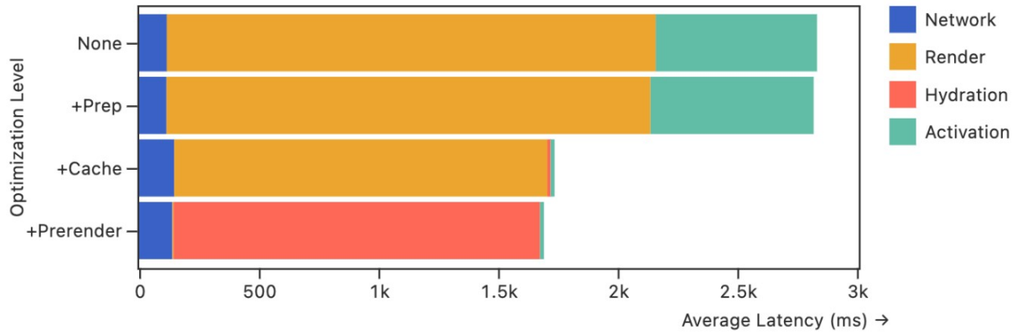



Figure 4: สัดส่วนเวลาเฉลี่ยจาก 5 กรณีทดสอบ ที่ขนาดข้อมูล 10 ล้านแถว (หน้า PDF 4)

## ลำดับเหตุการณ์การแสดงผลและการเปิดระบบโต้ตอบ

**1. ภาพเริ่มต้นปรากฏ (TTR ลดลง 83.7%)** → **2. เริ่มคอมโพเนนต์ Client Hydration** → **3. พร้อมรับอินพุต (TTA ลดลง 33.3%)**

ผังสรุปลำดับเหตุการณ์จาก Section 4.3 (แสดงขั้นตอน ไม่ใช่แกนเวลา Benchmark)

## การจำแนกประเภทมุมมองและข้อจำกัด

### 1 มุมมองแบบสถิต (Static Views)

ส่งเฉพาะผลลัพธ์ภาพที่เรนเดอร์สำเร็จ
ไร้สคริปต์ ไม่ต้องเสียต้นทุน Hydration

### 2 มุมมองแบบโต้ตอบ (Interactive Views)

ส่งภาพแรกพร้อมสคริปต์ JavaScript
ต้องเติมแคชและผูก Event ก่อนใช้งาน

### 3 คอขวด Hydration ฝั่งไคลเอนต์

ใช้เวลาเกือบเท่าการเรนเดอร์สดจากศูนย์
Hydration ที่ละเอียดขึ้นเป็นแนวทางวิจัยต่อ

ที่มา: Sections 4.3 และ 5 (หน้า PDF 3–4)

# ผลทดลองและการชั่งน้ำหนักทรัพยากร

## การเปรียบเทียบ TTR และ TTA ตามขนาดข้อมูล

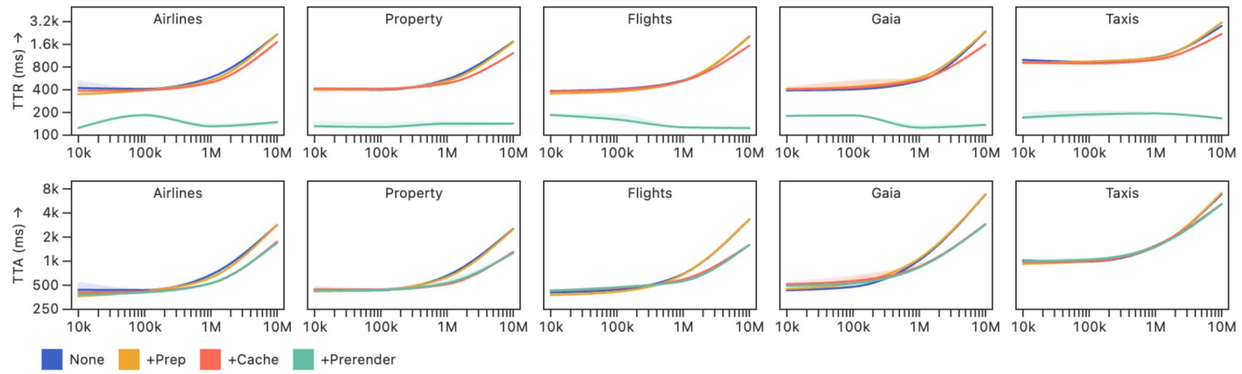



Figure 3: เส้นมัธยฐาน (Median) และแรเงา IQR จากข้อมูล 10k–10M แถว (หน้า PDF 4)

## การเปลี่ยนแปลงสัมพัทธ์เทียบ Baseline

| ระดับ | ΔSC % | ΔTTR % | ΔTTA % |
|---|---|---|---|
| +prep | 0.0 ± 0.0 | −0.6 ± 4.0 | −1.0 ± 2.5 |
| +cache | 21.1 ± 29.8 | −16.6 ± 1.3 | −32.3 ± 8.9 |
| +most | 21.2 ± 29.9 | −83.7 ± 2.1 | −33.3 ± 9.6 |

Table 1: ค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน (Mean ± σ) ข้ามทุกชุดทดสอบ แถว +most คือชุดรวมทุกกลยุทธ์ (ในเนื้อหาเปเปอร์ใช้ชื่อ +prerender)

**−83.7%**

**เวลาจนภาพปรากฏ (TTR)**

σ = 2.1% · ลด Perceived Latency ชัดเจน

**−33.3%**

**เวลาพร้อมโต้ตอบ (TTA)**

σ = 9.6% · แคชช่วยตัดเวลาหลัก (−32.3%)

**+21.2%**

**พื้นที่จัดเก็บเพิ่มเติม (SC)**

σ = 29.9% · แลกกับความเร็วในการโหลด

สภาพแวดล้อม: MacBook Pro (M1 Pro, RAM 16 GB), Node 22.9, Local Server, DuckDB-WASM

**ผลต่างการลด TTA: 33.3 − 32.3 = 1.0 จุดเปอร์เซ็นต์ เมื่อใช้ Baseline เดียวกัน**

ที่มา: Figure 3 และ Table 1 (หน้า PDF 4) · ค่าเฉลี่ยข้าม 5 รูปแบบภาพและขนาดข้อมูล 10k–10M แถว

# ผลงานหลักและข้อจำกัดของบทความ

## ผลงานหลักของบทความ

### 1 กลไกช่วงเผยแพร่ที่ใช้กับระบบจริง

เตรียมข้อมูล ผลคิวรี และภาพเริ่มต้นผ่าน Mosaic Publisher
ใช้ตัวกลางคิวรีร่วมกับการแก้ข้อกำหนดเฉพาะจุด

### 2 กรอบประเมิน TTR / TTA / SC

แยกเวลาจนภาพครบจากเวลาพร้อมรับอินพุต
พร้อมรายงานต้นทุนพื้นที่ที่เพิ่มขึ้น

### 3 ทดสอบแบบเพิ่มระดับการปรับปรุง

ภาพข้อมูล 5 รูปแบบ ที่ข้อมูล 10k–10M แถว
ทำให้เห็นว่าแคชกับภาพล่วงหน้าช่วยคนละช่วง

## ข้อจำกัดจากการอ่าน

### 1 สภาพแวดล้อมการทดสอบจำกัด

MacBook Pro M1 Pro และ Local Server
ยังไม่ได้ประเมินมือถือหรือเครือข่ายระยะไกล

### 2 ต้นทุน Hydration และพื้นที่

ยังเริ่มคอมโพเนนต์ฝั่งไคลเอนต์ตามปกติ
SC เพิ่มเฉลี่ย 21.2% แต่ σ สูงถึง 29.9%

### 3 ความสดและการรายงานการทดลอง

ข้อมูลเปลี่ยนต้องสร้างทรัพยากรใหม่
PDF ไม่ระบุจำนวนรอบทดลองซ้ำ

ที่มา: Sections 3–5 (หน้า PDF 2–4) · ข้อจำกัดสังเคราะห์จากขอบเขตหลักฐาน

# ข้อสรุปและแนวทางวิจัยต่อ

## ข้อสรุปจากการอ่าน

### 1 แคชลดงานก่อนพร้อมโต้ตอบ

+cache ลด TTA เฉลี่ย 32.3% เทียบ none
เป็นส่วนหลักของการลด TTA ในชุดทดสอบ

### 2 ภาพล่วงหน้าลดเวลารอเห็นภาพ

ชุดรวมลด TTR 83.7% และ TTA 33.3%
ภาพมาเร็ว แต่ยังเหลือต้นทุน Hydration

### 3 ผลด้านความเร็วมีต้นทุนพื้นที่

ชุดรวมเพิ่ม SC เฉลี่ย 21.2% ± 29.9
ประโยชน์ต้องพิจารณาตามเงื่อนไขของระบบ

## แนวทางที่ทีมวิจัยเสนอ

### 1 Hydration ที่ละเอียดขึ้น

ลดงานเริ่มคอมโพเนนต์ฝั่งไคลเอนต์
เพื่อลดต้นทุนที่ยังใกล้การเรนเดอร์เต็มชุด (§5)

### 2 Hybrid และ Incremental Regeneration

ผสานการปรับปรุงช่วงเผยแพร่กับรันไทม์
และสร้างทรัพยากรใหม่เฉพาะส่วน (§6)

### 3 เลือกการตั้งค่าโดยอัตโนมัติ

พัฒนาวิธีเลือกระดับการปรับปรุงจากข้อมูล
และข้อกำหนดภาพ แนวทางนี้ยังไม่ได้ทดลอง (§6)

ที่มา: ซ้าย Table 1 / Figure 4 · ขวา Sections 5–6 (หน้า PDF 4) · แนวทางวิจัยต่อยังไม่ได้ประเมิน